

Software Framework of Autonomous Mobile Robots on Isaac Sim and ROS

1st Mahfud Jiono

*College of Mechanical and Electrical Engineering
National Taipei University of Technology
Taipei 10608, Taiwan
mahfud.jiono.ft@um.ac.id*

2nd Hsien-I Lin*

*Senior Member, IEEE
Institute of Electrical and Control Engineering
National Yang Ming Chiao Tung University
Hsinchu 30010, Taiwan
sofin@nycu.edu.tw*

Abstract—The purpose of this research is to address the increasing significance of autonomous mobile robots (AMR) in military applications. Several researchers have made significant investments in AMR development, recognizing the urgency of enhancing their capabilities in preparation for potential threats in factory environments. The research aims to integrate key technologies essential for polishing robots in the factory using Isaac Sim and the ROS framework. This includes the use of AI and sensor fusion technology for robot movement, as well as for detecting and tracking obstacles, target objects, and the formation control of the AMR itself. The outcomes of our experiments have been promising. We have effectively defined and executed a sequence of arm robot positions. The robots start from the initial position, proceed to the predetermined second point, and subsequently advance to the position goal point before returning to their starting point. Concerning mobile robot movement control, we have established target positions to demonstrate the robot's capabilities.

Index Terms—AMR robot, Robot formation control, Isaac Sim simulation, ROS framework

I. INTRODUCTION

The goal of developing Autonomous Mobile Robots (AMRs), which integrate mobile and ARM robot capabilities for outdoor scenarios, is to gain an advantage in situational awareness, quick decision-making, the survivability of friendly forces, and the ability to neutralize adversary forces while maintaining visualization through simulation programs. Engineering teams can utilize advanced robot simulators like Isaac Sim [1] to develop and test these robots. AMRs must address various challenges in unknown environments, including traversing rough terrains and avoiding obstacles like holes, which can be monitored within the simulation environment. To tackle these difficulties, extensive analyses have been conducted to identify optimal scenarios for AMRs to operate effectively without encountering obstacles. Furthermore, we have considered the impact of adverse weather conditions, such as rain and fog, on sensor readings.

Regarding trajectory planning, local planning poses more difficulties than global planning and navigating successfully in factory environments. The motion planner's input data comprises the AMR's initial and target poses, derived from an

IMU installed on the robot [2] [3], which provides the x, y, and z coordinates, along with its rotation. Furthermore, the planner considers the end-effector position of the robotic arm when the mobile robot is at varying heights. Due to their considerable task flexibility and adaptability, arm robots also introduce intricate challenges in modeling, planning, and control [4]. These challenges vary based on whether the robot's end-effector makes contact with the external environment, such as handling an object. The system can be divided into two categories: the first category pertains to non-contact tasks, where the robot can trace the intended path within a structured environment, and the second category encompasses contact tasks, including coordinated handling, surface polishing, automobile polishing, and more.

To ensure secure interaction between the robot and its surroundings, the hybrid approach of combining position and force control is extensively embraced in interactive control systems for dual-arm collaborative robots [5]. This concept involves utilizing force control tactics within confined parameters and position control methods within unrestricted parameters. Another technique, applied in cases involving closed-chain kinematics and uncertain dynamics, represents a more advanced instance of the hybrid position/force control strategy [6]. Ravindran et al. [7] introduced a formation challenge focused on dynamically positioning safeguarding robots around a platoon. The outcomes highlight the success of their approach in realizing autonomous monitoring of mobile robot platoons, delivering positive results across various scenarios using ROS and Gazebo Software.

However, achieving accurate control performance for AMRs in uneven environments presents significant challenges. The main contribution of this paper is the introduction of a cooperative controller that effectively connects real-world operations with simulations. By integrating Isaac Sim and the ROS environment, our approach is designed to reduce errors and enhance overall performance. The structure of this paper unfolds as follows: Section II delves into the related work, while Section III presents the simulation outcomes of the autonomous positioning robots approach. Section IV thoroughly analyzes the results derived from the simulations. Finally, Section V offers concluding remarks on the research.

* Corresponding author

II. RELATED WORK

Numerous simulators have been created to replicate the behavior of aerial robots across diverse tasks, such as navigation, mapping, and control. One of these simulators is based on Gazebo [8] and encompasses various multi-rotors equipped with RGB-D sensors. Another simulator, Airsim [9], is constructed using the Unreal Engine, offering a photorealistic environment. This simulator supports hardware and software integration for a specific number of simulated robots. Flightmare [10] introduces the capability to parallelly simulate a substantial number of robots.

Robot formation entails the orchestrated positioning of multiple robots in a predetermined pattern or arrangement to attain an objective or optimize task performance. The forthcoming landscape sees robots assuming pivotal roles in aiding humans in perilous endeavors [11], including military endeavors. In such scenarios, robots often collaboratively operate alongside human soldiers. A recurring application in military contexts involves robots providing auxiliary support to infantry platoons during extended marches. Through the adoption of precisely engineered formations with automated positioning, these robots assume a protective role for the platoon, significantly enhancing their overall safety quotient.

NVIDIA's Isaac Gym [12] provides a simulation framework accelerated by GPUs, well-suited for robot learning efforts. This simulator is widely used for emulating robots with multiple links and joints [13]. While basic simulation environments have been created for aerial robots in alignment with Isaac Gym [12], these models require improved accuracy or interfaces that enable controlling motor forces while neglecting the influence of torque generated by motors on the robot's structure. These environments also lack support for advanced high-level reference tracking interfaces.

As a solution to harness GPU-enabled simulation capabilities, our simulator is constructed upon the foundation of the Isaac Gym simulator. We also introduce GPU-based geometric attitude and velocity controllers, thus broadening the scope of controllable inputs and expanding the simulator's applicability to a more diverse array of use cases. This enhancement empowers the simulator's potential for real-world training scenarios, bridging the gap between simulation and reality.

ROS [14] permits the creation of dependable software for controlling and navigating robots by leveraging its comprehensive libraries. These libraries provide software abstraction as well as drivers for prevalent robot sensors and actuators. For instance, libraries supporting tasks like computer vision, localization, and mapping are integrated. Gazebo [15] furnishes an advanced 3D virtual testing environment that includes widely-used robots characterized by highly accurate physics representation. Within the scope of algorithm development, ROS facilitates the utilization of both the C++ and Python programming languages.

III. SIMULATION ENVIRONMENT DESIGN

In the pursuit of AMR development for surface polishing in factory environments, the focus lies on achieving intelli-

gent and unmanned environmental detection and exploration through the implementation and simulation environment of autonomous vehicles. These AMRs enable cost-effective large-scale environmental detection by integrating artificial intelligence, environmental perception, communication, and power systems (such as fully electric or hybrid setups). They encounter challenges in trajectory planning and navigation due to the unpredictable and uneven environment.

We have developed the AMR simulation environment integrated with the tensor-based parallelization framework offered by NVIDIA's Isaac Sim simulator [1]. AMRs face unknown environmental challenges, such as traversing rough terrains, and on-road challenges, such as avoiding obstacles. The data is efficiently transmitted to the control center using technologies to ensure real-time monitoring of the autonomous mobile robot's status. By combining cutting-edge technologies and capabilities, these advancements contribute to the successful deployment of autonomous mobile robots in challenging environments, enhancing environmental awareness and exploration capabilities while maintaining seamless communication with the control center.

Moreover, an open-source machine learning framework leverages deep learning via Tensorflow or Pytorch for training agents using techniques like deep reinforcement learning. Isaac Sim, NVIDIA's latest scalable robotics simulator, excels in generating synthetic photorealistic data within physically accurate environments. Its purpose is to facilitate the development, testing, and management of AI-based robots. This simulator prioritizes realistic GPU-powered physics, effectively tackling common robotics challenges such as manipulation, navigation, and the creation of synthetic training data. With its modular structure and the ability to conduct sim2real experiments, this simulator encompasses vital attributes found in the simulators mentioned earlier, positioning itself as an advanced tool for training robots in deep learning methodologies.

To display multiple sensors in RViz, ensuring synchronization and correct timestamping of messages becomes crucial. The simulator enables comprehensive sensor visualization by incorporating a Lidar sensor, utilizing a Lidar Rostopic, and displaying multiple sensors in RViz. The stereo GEM (Geometry Engine for Mobile Robots) generates a point cloud, providing valuable information about the objects within the cameras' field of view and their spatial positions in the environment. This information updates the navigation node, allowing the robot's path to be adjusted if a potential collision is detected. Overall, this simulator combines advanced physics simulation, sensor integration, and path planning capabilities, making it a powerful tool for training robots and simulating real-world scenarios.

We introduced a flexible asset management class that empowers users to load URDF files describing obstacles from a designated folder structure. This methodology, in sync with NVIDIA Isaac Gym, characterizes assets as mesh entities within the simulation, encompassing links, joints, and other physical attributes. While our present implementation employs URDF files to represent assets, extending support to other

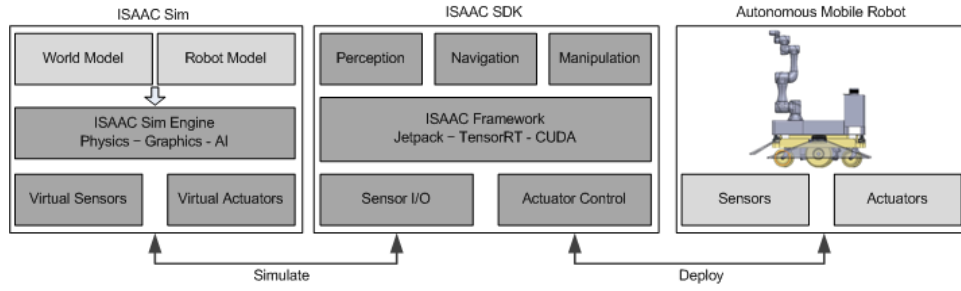


Fig. 1. System architecture of the AMR.

formats is seamless. The process of asset selection and loading transpires randomly for each environment. A specified count of distinct obstacle meshes is drawn from user-defined obstacle classes, ensuring diversity and avoiding the recurrence of identical obstacle meshes across environments.

In environments rich with obstacles, simulated robots can be outfitted with camera sensors capable of capturing an array of image types. The positions and orientations of these camera sensors can be configured and randomized. These images are amalgamated into a tensor accessible to users. Furthermore, we've adapted nonlinear geometric controllers tailored for aerial robots and, crucially, parallelized these controllers for GPU operation. This paves the way for furnishing high-level input commands to robots, employing interfaces reminiscent of those featured in prevalent flight controllers designed for multirotor aerial vehicles [16].

The simulator's structure is depicted in Fig. 1, illustrating the interaction between different modules. Robot state information, encompassing position, orientation, linear velocity, and angular velocity, is accessible to a high-level planner or learning-based framework. The simulator grants access to image tensors captured by onboard sensors, which can be utilized by user-defined navigation or learning methods. Additionally, privileged access to each obstacle's state aids in training learning-based approaches.

IV. AMR SPECIFICATIONS AND CONTROLLERS DESIGN

These AMR specifications and controllers enable cost-effective large-scale environmental detection by integrating artificial intelligence, environmental perception, communication, and power systems (such as full electric or hybrid setups) as described in the following section.

A. AMR Specifications

An encounter challenges in trajectory planning and navigation due to the unpredictable and uneven of factory environment. AMRs face unknown environmental challenges such as traversing rough terrains and on-road challenges such as avoiding obstacles like workers and etc. These robots are equipped with TM5-700 industrial robots, sensors, control systems, and onboard intelligence (Jetson TX2) to perceive their surroundings, make decisions, and execute actions accordingly. AMRs model as shown in Fig. 3 designed to navigate and interact with their environment to perform specific tasks or



Fig. 2. AMR Model.

achieve objectives. As an added contribution for generating procedural URDF models of simplified multi-linked tree-like objects, ready to be incorporated into the simulator that is mainly controlled by Jetson Tx2 as shown in Fig. 3 These procedurally generated trees can be customized in terms of length, diameters, and branching factors, empowering users to diversify collision avoidance training scenarios across forest-like environments. Hardware Specifications: (1) Capable of driving on general sidewalks and overcoming inclines. (2) Capable of operating in outdoor open spaces. (3) Maximum speed: 4 km/hr (inclusive). (4) Estimated dimensions of the vehicle body: approximately 65cm (width) x 95cm (length) x 50cm (height). (5) Provided equipment includes an electrical control box, a fixed base for the robotic arm (TM5-700), a controller base for the robotic arm, and a sensor mounting base. (6) Maximum payload weight: 100 kg for uphill transportation. (7) Additional Sensor (camera and Lidar) (8) PLC specification for vehicle control: FX5UC-32MT/DS-TS. (9) Manual charging for the batteries, charger specification: 24V 7A. (10) Vehicle battery specification: Lithium Iron Phosphate (LiFePO4) 24V 50AH. (11) Robotic arm battery specification: Lithium Iron Phosphate (LiFePO4) 48V 35AH. (12) TM5-700 industrial robot mounted on the AMR

B. AMR Differential Drive Controller

The differential drive kinematic model stands as a simplified mathematical framework employed to depict the movement of a two-wheeled robot or vehicle, akin to a mobile robot or a basic robotic car. This model finds extensive application

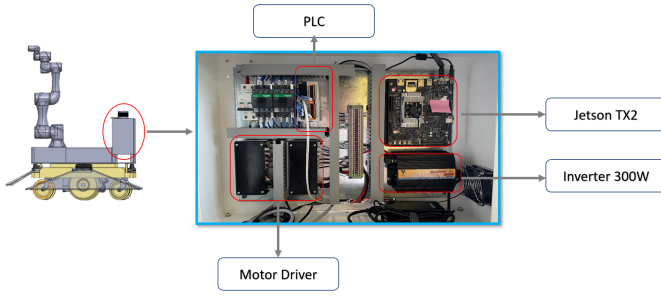


Fig. 3. Model and controller of the AMR.

within the domains of robotics and control theory, facilitating the comprehension of fundamental behaviors exhibited by such systems. Within the context of the differential drive kinematic model, the vehicle is posited to possess a pair of wheels situated upon a shared axis. Governing the motion involves the independent manipulation of the rotational speed of each wheel. Consequently, the vehicle attains the capability to effectuate forward and backward movement, execute precise in-place rotations, or traverse trajectories encompassing curved paths. The principal constituents integral to the differential drive kinematic model comprise; Wheel Radii: This model postulates distinct radii for both wheels, which are denoted as r_{left} and r_{right} for the left and right wheels, respectively. Wheel Speeds: The rotational speed of the left wheel is represented as ω_{left} , while the corresponding speed for the right wheel is indicated as ω_{right} . These speeds collectively determine the rotational rates of the individual wheels. Vehicle Velocity (Linear and Angular): The linear velocity "v" of the vehicle emerges as the mean of the linear velocities associated with both wheels. This quantity is established through the ensuing equation:

$$v = (r_{left} * \omega_{left} + r_{right} * \omega_{right}) / 2 \quad (1)$$

The angular velocity " ω " of the vehicle is calculated using the difference in linear velocities of the two wheels and the distance between the wheels (wheelbase "L"):

$$\omega = (r_{right} * \omega_{right} - r_{left} * \omega_{left}) / L \quad (2)$$

substituting Equations 1 and 2, the resultant expression for the updated position and orientation is represented as Equation (3).

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} (r \cos \theta) / 2 & (r \cos \theta) / 2 \\ (r \sin \theta) / 2 & (r \sin \theta) / 2 \\ 2r / L & -2r / L \end{bmatrix} \begin{bmatrix} \omega_l \\ \omega_r \end{bmatrix} \quad (3)$$

By integrating Equation (3), and subsequently adding it to the initial position, the ultimate position is characterized by Equations (4).

$$\begin{aligned} x &= x_0 + \Delta x = x_0 + \int \dot{x} dt \\ y &= y_0 + \Delta y = y_0 + \int \dot{y} dt \\ \theta &= \theta_0 + \Delta \theta = \theta_0 + \int \dot{\theta} dt \end{aligned} \quad (4)$$

By controlling the wheel speeds (ω_{left} and ω_{right}), the robot can follow different types of trajectories, including straight lines, arcs, and turns. The differential drive kinematic model assumes that our wheels are perfect wheels with small slippage and neglects factors like wheel deformation, friction, and external forces. The differential drive kinematic model is a foundational concept in robotics, often used for initial planning and control algorithms. However, for more accurate representations and control in real-world scenarios, additional factors and dynamics, such as wheel slippage, motor dynamics, and external disturbances, must be considered by implementing advanced control strategies. Implement advanced control strategies, such as model predictive control or adaptive control, that adjust the robot's behavior based on real-time feedback and the evolving environment.

C. AMR-ROS Framework

The main framework employed in our AMR is constructed upon the ROS architecture. ROS, an open-source middleware framework designed for Linux, is tailored for modular applications in robot contexts. ROS's software architecture hinges on the concept of a package, which acts as a repository for ROS runtime execution programs, known as nodes. Additionally, a package may house ROS-independent libraries, datasets, configuration files, third-party software, or any other software components that warrant organized grouping, as depicted in Fig. 4. The prime aim of these packages is to offer accessible functionality in an organized manner, thereby fostering software reuse across diverse projects. Through this structure, coupled with the principles of object-oriented programming, packages serve as modular building blocks, seamlessly interlocking to achieve desired outcomes.

Typically, packages adhere to a shared structure, encompassing vital elements such as package manifests, message types, service types, headers, executable scripts, build files, and runtime processes. By adhering to this modular approach and conforming to standardized package formats as displayed in Fig. 4, ROS facilitates the efficient development and integration of software components within the AMR system. This design philosophy elevates the reusability, maintainability, and scalability of code, fostering harmonious collaboration among distinct packages to effectively fulfill AMR's objectives.

For the purpose of AMR robot control within ROS, the creation of a ROS node becomes necessary. This node subscribes to a topic supplying the desired position and issues command to maneuver the robot accordingly, as illustrated in Fig. 5. The specified task entails defining the desired position and orientation of the coordinate frame linked to the robot's end-effector relative to the base frame, referred to as task-space variables. Feedback from joint or task-space variables is commonly harnessed to govern end-effector motion. Joint-based control necessitates solving the inverse kinematics problem, translating the intended task-space trajectory into a corresponding joint trajectory.

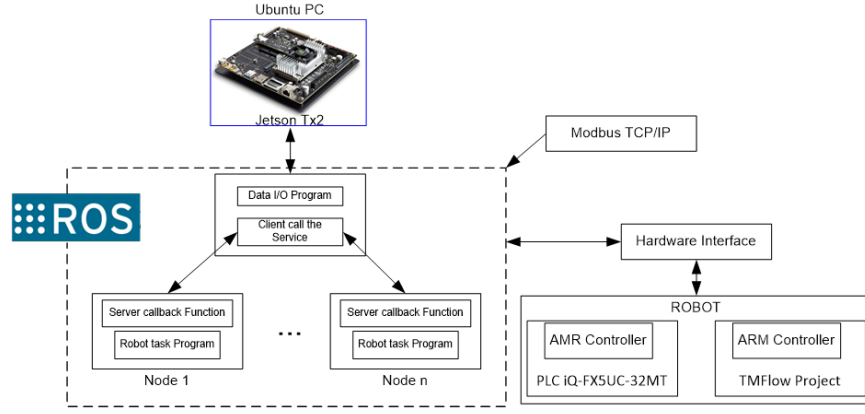


Fig. 4. ROS framework of the AMR.

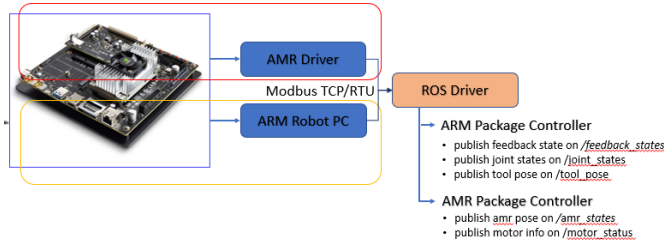


Fig. 5. The communication and package of the AMR.

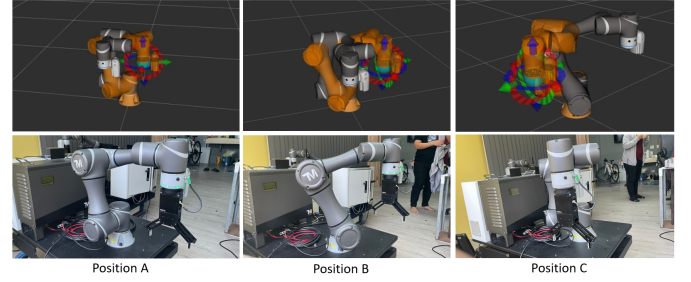


Fig. 6. TM5-700 Position Controller through the MoveIt ROS Environment

V. EXPERIMENTAL RESULTS

In our AMR system, we constructed the architecture on top of the ROS framework, featuring nodes dedicated to communication with the TM5-700 robotic arm via the MoveIt ROS controller. This communication encompasses data transmission, including robot states, images sourced from the eye-in-hand camera, and URDF models for various robot arms, facilitated through TMflow. The MoveIt platform, a cornerstone of robotic manipulation within ROS, embodies cutting-edge advancements across motion planning, manipulation, 3D perception, kinematics, control, and navigation domains. We've employed two TM5-700 industrial robots to establish a dual-arm robotic system, where a coordinated transport experiment involving both arms is tested within this simulation environment. The robot under consideration embodies a redundant manipulator endowed with 6 degrees of freedom. These surplus degrees of freedom enrich the manipulator's joint space configurations during motion, augmenting trajectory planning's versatility and diversity.

As for the testing results, we have defined 3 different positions (A, B, and C) as shown in Fig.6. We can see that the robot performs a point-to-point (PTP) motion to move the TM robot to specific joint angles as shown in Table. I. It starts at the initial position A, then moves to position B as defined earlier, and then moves again to position C. After that, it returns to position A as its initial position. Simultaneously, we can observe real-time movement and monitoring results through the MoveIt.

TABLE I

PERFORM A POINT-TO-POINT (PTP) MOTION TO MOVE THE TM ROBOT TO SPECIFIC JOINT ANGLES.

Join Target	Specific joint angles
joint_goal_start	[3.319, 0.163, 0.478, 1.010, 1.537, 0.137]
joint_goal_a	[3.603, -0.194, 1.355, 0.490, 1.571, 0.420]
joint_goal_b	[2.939, -0.082, 1.986, -0.270, 1.509, -0.241]
joint_goal_c	[3.974, -0.073, 1.283, 0.4456, 1.593, 0.790]
joint_goal_start	[3.319, 0.163, 0.478, 1.010, 1.537, 0.137]

To control the AMR robot using Modbus TCP/IP, the AMR has specific Modbus registers that allow control of its movements, such as speed, direction, and other relevant parameters, as shown in Table. II. Refer to the robot's documentation for the correct register addresses and data format. The initial parameter can act as the Modbus master to send commands to the robot. Use the Modbus library parameter to write commands to the appropriate Modbus registers. Depending on the desired actions, these commands will instruct the robot to move, turn, stop, etc. The common construction of a differential drive ground mobile robot consists of two wheels mounted on a common axis. Each wheel can be controlled independently to move either forward or backward. The turning angle, which refers to the point of the instantaneous center of rotation (ICR), can be achieved by varying the angular speed of each wheel. The kinematic model of the ground mobile robot can be represented as described in the previous section.

Furthermore, for the results of the mobile robot movement

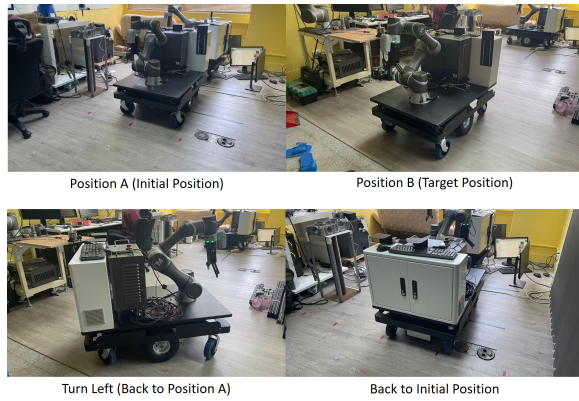


Fig. 7. AMR Position Controller through the MoveIt ROS Environment

TABLE II
AMR CONTROLLER CONFIGURATIONS.

Protocol Format	Modbus TCP/IP
IP Address	192.168.1.1
Port Number	502
Register Address	Instructions
D606	Bit0:Start Bit1:Stop Bit2:Reset Bit3:Breaks Bit4:Alarm in
D610	X-axis offset(-4000 – 4000)
D611	Y-axis offset(-8000 – 8000)

control, we have also initialized its target position. As seen in Fig. 7, the AMR will start moving from position A, moving straight at a constant speed towards position B. After reaching its target position (position B), the robot will rotate 180° to return to its initial position (position A).

VI. CONCLUSION

In our implemented Autonomous Mobile Robot (AMR) system, we have developed a robust framework using the Robot Operating System (ROS). This framework encompasses essential communication nodes that interface with the TM5-700 robotic arm through the MoveIt ROS controller. These nodes facilitate the exchange of crucial data, including real-time robot states, images captured by the eye-in-hand camera, and Unified Robot Description Format (URDF) models that support diverse robot arms via TMflow.

The outcomes of our experiments have been promising. We have effectively defined and executed a sequence of positions (A, B, and C). The robots commence from the initial position A, proceed to the predetermined point B, and subsequently advance to position C before returning to their starting point A. In terms of mobile robot movement control, we have established target positions to demonstrate the robot's capabilities. The AMRs can move from position A, progress linearly to position B at a consistent velocity, perform a 180° rotation upon reaching position B, and ultimately return to their initial position A.

As future work, we are dedicated to integrating obstacle avoidance functionalities within the robots. Additionally, we are resolute in actualizing these advancements on tangible

ground mobile robots. The meticulous technical refinements and pragmatic validation tests developed throughout this research endeavor will persist as we progress forward.

ACKNOWLEDGMENT

Many thanks to the National Science and Technology Council, Taiwan for the Grant NSTC 111-2221-E-A49 -192 -MY3

REFERENCES

- [1] "Isaac sim - robotics simulation and synthetic data generation", July 2023, [online] Available: <https://developer.nvidia.com/isaac-sim>.
- [2] I. Kecskés, R. Bacsá, P. Odry, Z. Vizvári and A. Toth, "Reproducibility analysis of mobile robots movement by IMU sensor measurements," 2021 IEEE 15th International Symposium on Applied Computational Intelligence and Informatics (SACI), Timisoara, Romania, 2021, pp. 000113-000118, doi: 10.1109/SACI51354.2021.9465627.
- [3] M. Christian et al., "Application of Deep Learning to IMU sensor motion," 2019 SoutheastCon, Huntsville, AL, USA, 2019, pp. 1-6, doi: 10.1109/SoutheastCon42311.2019.9020363.
- [4] A. T. Jawad, N. S. Ali, A. N. Abdullah and N. H. Alwash, "Design of Adaptive Controller for Robot Arm Manipulator Based on ANN with Optimized PID by IWO Algorithm," 2021 International Conference on Advanced Computer Applications (ACA), Maysan, Iraq, 2021, pp. 107-111, doi: 10.1109/ACA52198.2021.9626781.
- [5] A. Zahavi, S. N. Haeri, D. C. Liyanage and M. Tamre, "A Dual-Arm Robot for Collaborative Vision-Based Object Classification," 2020 17th Biennial Baltic Electronics Conference (BEC), Tallinn, Estonia, 2020, pp. 1-5, doi: 10.1109/BEC49624.2020.9277067.
- [6] Yi Ren, Zhengsheng Chen, Yechao Liu, Yikun Gu, Minghe Jin, Hong Liu, "Adaptive hybrid position/force control of dual-arm cooperative manipulators with uncertain dynamics and closed-chain kinematics," Journal of the Franklin Institute, Volume 354, Issue 17, 2017, Pages 7767-7793, ISSN 0016-0032, doi: <https://doi.org/10.1016/j.jfranklin.2017.09.015>.
- [7] R. Ravindran, J. P. Mills and M. Krishnan, "Autonomous Multi-Robot Platoon Monitoring," 2018 IEEE 61st International Midwest Symposium on Circuits and Systems (MWSCAS), Windsor, ON, Canada, 2018, pp. 328-331, doi: 10.1109/MWSCAS.2018.8624037.
- [8] D. Chikurtev, "Mobile Robot Simulation and Navigation in ROS and Gazebo," 2020 International Conference Automatics and Informatics (ICAI), Varna, Bulgaria, 2020, pp. 1-6, doi: 10.1109/ICAI50593.2020.9311330.
- [9] S. Shah et al., "AirSim: High-fidelity visual and physical simulation for autonomous vehicles," Proc. Field Serv. Robot.: Results 11th Int. Conf., pp. 621-635, 2017.
- [10] Y. Song, S. Naji, E. Kaufmann, A. Loquercio and D. Scaramuzza, "Flightmare: A flexible quadrotor simulator", Conference on Robot Learning, 2020.
- [11] M. -F. R. Lee, H. P. M. Willybrordus, Sukamto, S. Hassen and A. Nugroho, "Formation Control for Mobile Robot using Fuzzy - PI Controller," 2020 International Conference on Advanced Robotics and Intelligent Systems (ARIS), Taipei, Taiwan, 2020, pp. 1-6, doi: 10.1109/ARIS50834.2020.9205789.
- [12] V. Makovychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa et al., "Isaac Gym: High performance GPU-based physics simulation for robot learning," 2021.
- [13] Tsai, Chia-Hao, Jun-Ji Lin, Teng-Feng Hsieh, and Jia-Yush Yen, "Trajectory Control of An Articulated Robot Based on Direct Reinforcement Learning," Robotics 2022, 11, 116. doi: <https://doi.org/10.3390/robotics11050116>.
- [14] M. Quigley et al., "ROS: An open-source robot operating system", Proc. ICRA Workshop Open Source Softw., 2009.
- [15] K. Takaya, T. Asai, V. Kroumov and F. Smarandache, "Simulation environment for mobile robots testing using ROS and Gazebo," 2016 20th International Conference on System Theory, Control and Computing (ICSTCC), Sinaia, Romania, 2016, pp. 96-101, doi: 10.1109/IC-STCC.2016.7790647.
- [16] A. L. Silva and D. A. Santos, "Fast Nonsingular Terminal Sliding Mode Flight Control for Multirotor Aerial Vehicles," in IEEE Transactions on Aerospace and Electronic Systems, vol. 56, no. 6, pp. 4288-4299, Dec. 2020, doi: 10.1109/TAES.2020.2988836.